Examen Parcial I
(Decodificador y Display)

Alex Rafael Soriano Severino
Mecatronica
ITLA
La Caleta, Santo Domingo
20240684@itla.edu.do

En resumen— En esta práctica se diseñó e implementó un circuito combinacional tipo decodificador capaz de mostrar en un display de 7 segmentos (cátodo común) las iniciales y los dígitos correspondientes a la fecha de nacimiento. El sistema fue programado en lenguaje HDL (Verilog) con descripción estructural e implementado sobre una FPGA utilizando el entorno GoWin.
Se elaboró la tabla de verdad para once combinaciones correspondientes a la secuencia "ARS82364075", donde algunos dígitos fueron modificados para evitar repeticiones, según las indicaciones del mandato. Las funciones lógicas se simplificaron mediante Mapas de Karnaugh, se generó el diagrama lógico y se verificó el comportamiento del circuito mediante simulación y pruebas físicas. Finalmente, se comprobó que las salidas del display coincidían con los resultados teóricos y que la visualización era estable al modificar manualmente las entradas a través de los DIP switches.

Abstract— In this practice, a combinational decoder circuit was designed and implemented to display the initials and birth date on a 7-segment (common cathode) display. The system was developed using HDL (Verilog) in a structural description approach and implemented on an FPGA using the GoWin IDE. A truth table with eleven combinations representing the sequence "ARS82364075" was designed, with minor digit adjustments to avoid repetition according to the project instructions. The logic functions were simplified using Karnaugh maps, and the circuit was verified through both simulation and FPGA testing. Results confirmed that the theoretical truth table matched the experimental behavior, and the manual DIP switch control correctly activated the desired display segments.

Palabras claves— Electrónica digital, decodificador, display de 7 segmentos, FPGA, Verilog estructural, circuito combinacional.

I. INTRODUCCION

El propósito de esta práctica fue diseñar un circuito combinacional tipo decodificador que permitiera visualizar en un display de 7 segmentos las iniciales y fecha de nacimiento, siguiendo una secuencia personalizada de once combinaciones.
El diseño se realizó en lenguaje Verilog utilizando descripción estructural, lo que permitió comprender cómo las compuertas y módulos básicos pueden organizarse para generar una respuesta lógica determinada.

Aunque el mandato sugería automatizar las combinaciones mediante un contador, en este caso se optó por el control manual a través de DIP switches, simplificando la experimentación y facilitando la comprobación de cada símbolo en el display.
El circuito se diseñó para un display de cátodo común, donde los segmentos se activan mediante un nivel lógico alto (1). La secuencia mostrada fue ARS82364075, en la que algunos números se modificaron para cumplir con la restricción de no repetición establecida en el enunciado.

II. MARCO TEÓRICO

A. Decodificadores

Un decodificador es un circuito combinacional que convierte un conjunto de entradas binarias en un patrón único de salidas. Su función principal es traducir combinaciones digitales en señales que puedan representar letras, números u otras formas de información. En este caso, el decodificador traduce los valores de los DIP switches en los patrones de segmentos que deben encenderse para formar los caracteres requeridos.

B. Tabla de Verdad

La tabla de verdad es una herramienta que muestra todas las posibles combinaciones de entradas y las salidas resultantes. Es esencial para diseñar, verificar y comprobar el comportamiento de un circuito lógico.

C. Display de 7 Segmentos

El display de 7 segmentos está formado por siete LEDs dispuestos en forma de "8", etiquetados de a a g. Al controlar cada segmento de manera individual, pueden representarse números del 0 al 9 y algunas letras.
En un display de cátodo común, todos los cátodos de los LEDs están conectados a tierra, por lo que los segmentos se iluminan aplicando un nivel alto (1) en sus ánodos.
El patrón lógico que determina qué segmentos se encienden para cada símbolo es definido mediante una tabla de verdad.

D. Descripción Estructural en Verilog

La descripción estructural en Verilog consiste en construir el circuito a partir de módulos básicos, como si se conectaran compuertas físicas. En lugar de describir la función lógica con ecuaciones, se definen las interconexiones entre módulos, reflejando directamente la estructura del hardware. Este método resulta ideal para implementaciones en FPGA, ya que permite simular la arquitectura real del circuito.

III. MONTAJE EXPERIMENTAL

1. **Diseño Lógico:** Se elaboró la tabla de verdad con once combinaciones que corresponden a las letras y dígitos de la secuencia ARS82364075.
   Cada entrada binaria de 4 bits se asoció a un patrón específico de siete salidas que controlan los segmentos del display.

Las funciones de salida se simplificaron mediante Mapas de Karnaugh, obteniendo expresiones mínimas para cada segmento (a, b, c, d, e, f, g), lo que permitió optimizar el número de compuertas utilizadas en la descripción estructural.

2. **Simulación:** El circuito fue simulado en el entorno GoWin IDE y Multisim para los diagramas, utilizando el editor y simulador HDL integrados. Se analizaron las formas de onda y se verificó que cada combinación de entrada activara correctamente el patrón correspondiente a la letra o número esperado.
El diagrama lógico se generó en software, mostrando las interconexiones entre compuertas AND, OR y NOT que conforman el decodificador.

3. **Implementación en FPGA:** La implementación física se realizó en la FPGA, donde las entradas provenían de los DIP switches y las salidas se dirigían al display de 7 segmentos de cátodo común.
Cada cambio en los interruptores modificaba la combinación binaria de entrada, provocando la visualización de una letra o número distinto.
Adicionalmente, un conjunto de LEDs auxiliares mostró el valor binario equivalente de las entradas, permitiendo verificar visualmente la correspondencia entre el número binario y el carácter desplegado.

IV. FUNCIONAMIENTO DEL CIRCUITO

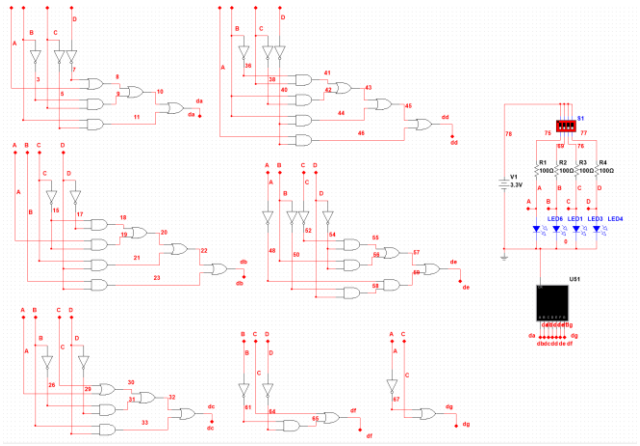
El circuito decodificador responde de forma directa a las combinaciones de los DIP switches.
Cada valor binario (desde 0000 hasta 1010) activa una configuración de segmentos específica, representando las letras y números de la secuencia ARS82364075.
Si se selecciona una combinación no definida en la tabla de verdad, el sistema apaga todos los segmentos del display, evitando confusión en la lectura.
En resumen, el circuito actúa como un traductor entre combinaciones binarias y representaciones visuales en el display, permitiendo recorrer de manera manual las once configuraciones establecidas.

V. RESULTADOS

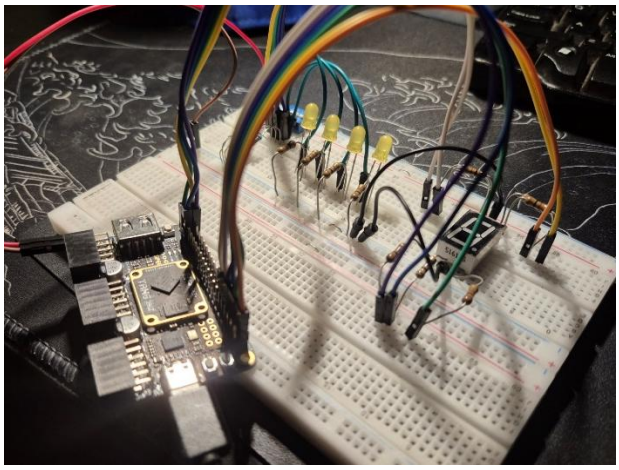
Durante las pruebas de simulación y la verificación física sobre el FPGA, las salidas mostraron coherencia total con los valores teóricos de la tabla de verdad.
Los caracteres se representaron de manera clara en el display y las transiciones entre combinaciones fueron estables y sin interferencias.
El funcionamiento del circuito confirmó que las funciones simplificadas obtenidas con Mapas de Karnaugh fueron correctas y eficientes, reduciendo la cantidad de compuertas requeridas.

El conjunto de LEDs adicionales permitió además visualizar en binario la combinación activa, facilitando la comprobación durante la práctica.

Diseño lógico (MULTISIM):



Físico:



VI. ANÁLISIS Y/O CÁLCULOS

Aquí calcularemos todos los datos solicitados en la práctica:

A. Tabla de Verdad

Tabla de Verdad											
A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	1	1	1	0	1	1	1	A
0	0	0	1	0	0	0	0	1	0	1	R
0	0	1	0	1	0	1	1	0	1	1	S
0	0	1	1	1	1	1	1	1	1	1	8
0	1	0	0	1	1	0	1	1	0	1	2
0	1	0	1	1	1	1	1	0	0	1	3
0	1	1	0	1	0	1	1	1	1	1	6
0	1	1	1	0	1	1	0	0	1	1	4
1	0	0	0	1	1	1	1	1	1	0	0
1	0	0	1	1	1	1	0	0	0	0	7
1	0	1	0	1	0	1	1	0	1	1	5
1	0	1	1	x	x	x	x	x	x	x	x
1	1	0	0	x	x	x	x	x	x	x	x
1	1	0	1	x	x	x	x	x	x	x	x
1	1	1	0	x	x	x	x	x	x	x	x
1	1	1	1	x	x	x	x	x	x	x	x

B. Mapas de Karnaugh

Salida (a)				
AB/CD	00	01	11	10
00	1	0	1	1
01	1	1	0	1
11	x	x	x	x
10	1	1	x	1

Salida (b)				
AB/CD	00	01	11	10
00	1	0	1	0
01	1	1	1	0
11	x	x	x	x
10	1	1	x	0

Salida (c)				
AB/CD	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	x	x	x	x
10	1	1	x	1

Salida (d)				
AB/CD	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	x	x	x	x
10	1	0	x	1

Salida (e)				
AB/CD	00	01	11	10
00	1	1	1	0
01	1	0	0	1
11	x	x	x	x
10	1	0	x	0

Salida (f)				
AB/CD	00	01	11	10
00	1	0	1	1
01	0	0	1	1
11	x	x	x	x
10	1	0	x	1

Salida (g)				
AB/CD	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	x	x	x	x
10	0	0	x	1

C. Funciones Lógicas

Estan ordenadas respectivamente a los k maps anteriores:

SoP
D' + A + B'C + BC'

SoP
C'D' + AC' + CD + BD

SoP
C + A + B'D' + BD

SoP
C'D' + AC' + CD + BD

SoP
C'D' + BD' + A'B'D

SoP
C + B'D'

SoP
A' + C

D. Programación en Gowin con verilog

```
module DECODIFICADOR(
    // Inputs and outputs declaration
    input A,B,C,D,
    output Sa,Sb,Sc,Sd,Se,Sf,Sg
);

    //Wires Declaration
    wire nwA, nwB, nwC, nwD; //not wires

    wire ow1, ow2, ow3, ow4, ow5, ow6, ow7,
         ow8, ow9, ow10, ow11, ow12, ow13,  //OR wires
         ow14, ow15, ow16, ow17, ow18, ow17_aux;

    wire aw1, aw2, aw3, aw4, aw5, aw6, aw7, //AND wires
         aw8, aw9, aw10, aw11, aw12;

    //NOT module declaration
    NOT IC_NOT(
         .NE1(A),               // not 1
         .NE2(B),               // not 2
         .NE3(C),               // not 3
         .NE4(D),               // not 4

         .NS1(nwA),
         .NS2(nwB),
         .NS3(nwC),
         .NS4(nwD)
    );

    //OR module declaration
    // IC1
    OR IC_OR1(
        .OE1(nwD),.OE2(A),   // or 1
        .OE3(ow1),.OE4(aw1),   // or 2
        .OE5(ow2),.OE6(aw2),   // or 3
        .OE7(aw3),.OE8(aw4),       // or 4

        .OS1(ow1),
        .OS2(ow2),
        .OS3(Sa),
        .OS4(ow3)
    );

    //IC2
    OR IC_OR2(
        .OE1(ow3),.OE2(aw5),       // or 5
        .OE3(ow4),.OE4(aw6),     // or 6
        .OE5(C),.OE6(A),   // or 7
        .OE7(ow5),.OE8(aw7),       // or 8

        .OS1(ow4),
        .OS2(Sb),
        .OS3(ow5),
        .OS4(ow6)
    );

    //IC3
    OR IC_OR3(
        .OE1(ow6),.OE2(aw8),   // or 9
        .OE3(aw9),.OE4(aw10),       // or 10
        .OE5(ow7),.OE6(aw11),       // or 11
        .OE7(ow8),.OE8(aw12),       // or 12

        .OS1(Sc),
        .OS2(ow7),
        .OS3(ow8),
        .OS4(Sd)
    );
```

```
    //IC4
    OR IC_OR4(
        .OE1(aw13),.OE2(aw14),       // or 13
        .OE3(ow9),.OE4(aw16),       // or 14
        .OE5(C),.OE6(aw17),       // or 15
        .OE7(nwA),.OE8(C),   // or 16

        .OS1(ow9),
        .OS2(Se),
        .OS3(Sf),
        .OS4(Sg)
    );

    //AND module declaration
    // IC1
    AND1 IC_AND1(
        .AE1(nwB),.AE2(C),   // and 1
        .AE3(B),.AE4(nwC),   // and 2
        .AE5(nwC),.AE6(nwD),   // and 3
        .AE7(A),.AE8(nwC),   // and 4

        .AS1(aw1),
        .AS2(aw2),
        .AS3(aw3),
        .AS4(aw4)
    );

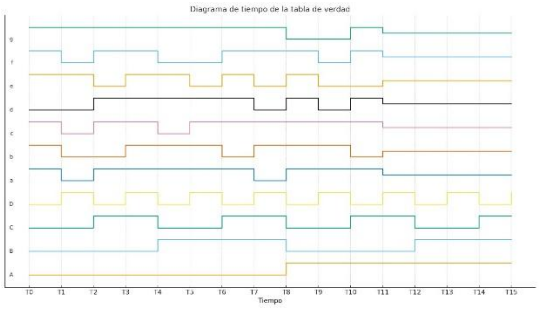
    // IC2
    AND1 IC_AND2(
        .AE1(C),.AE2(D),   // and 5
        .AE3(B),.AE4(D),   // and 6
        .AE5(nwB),.AE6(nwD),   // and 7
        .AE7(B),.AE8(D),   // and 8

        .AS1(aw5),
        .AS2(aw6),
        .AS3(aw7),
        .AS4(aw8)
    );

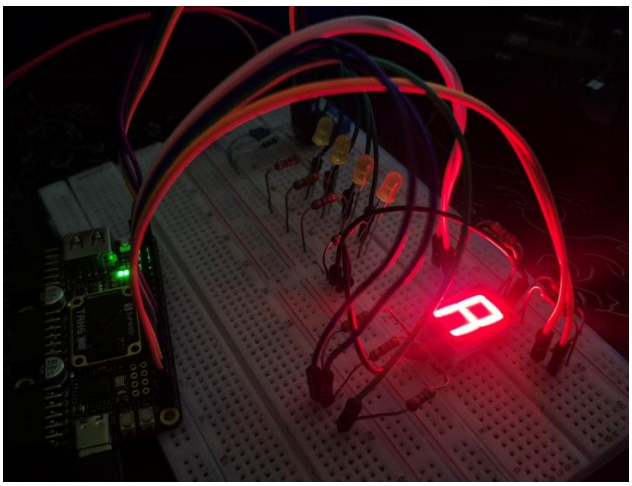
    // IC3
    AND1 IC_AND3(
        .AE1(nwB),.AE2(C),   // and 9
        .AE3(B),.AE4(nwC),   // and 10
        .AE5(B),.AE6(nwD),   // and 11
        .AE7(A),.AE8(nwD),   // and 12

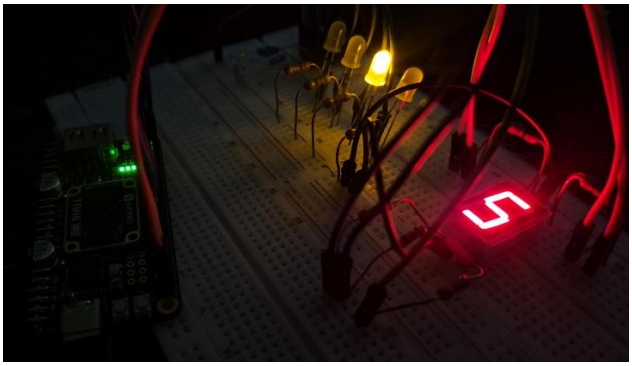
        .AS1(aw9),
        .AS2(aw10),
        .AS3(aw11),
        .AS4(aw12)
    );
```

E. Diagrama de Tiempo



F. Imágenes en físico (En funcionamiento)





VII. Conclusion

La práctica permitió diseñar, simular e implementar un decodificador funcional capaz de representar letras y números en un display de 7 segmentos.
El uso de Verilog estructural en el entorno GoWin facilitó el entendimiento de cómo se interconectan las compuertas lógicas para formar circuitos más complejos.
Aunque se sustituyó el contador por el control manual mediante DIP switches, el objetivo del ejercicio se logró completamente, mostrando de manera ordenada las once combinaciones correspondientes a la secuencia ARS82364075.

Se comprobó que el comportamiento del circuito físico coincidió con la simulación teórica, demostrando la efectividad del diseño combinacional y la utilidad del enfoque estructural en proyectos digitales basados en FPGA.

VIII. REFERENCIAS

https://en.wikipedia.org/wiki/Digital_electronics
https://naylampmechatronics.com/blog/39_resistencias-pull-up-y-pull-down.html
https://es.wikipedia.org/wiki/Sistema_de_numeraci%C3%B3n
https://es.wikipedia.org/wiki/C%C3%B3digo_binario